

L'intelligence sémantique au service du recrutement équitable :

DEVSPOT

Master Cyber 2024 - 2026

Trari Mehdi

Dufrénois Mélène

Sommaire

Introduction	3
Besoin ciblé et problématique	3
Motivation et opportunité	4
Positionnement par rapport à l'existant	5
Méthodologie et organisation	6
Conception de la solution	7
Réalisation / mise en œuvre	9
Tests, validation et limites	10
Limites et perspectives	11
Conclusion	12
Annexes	14

Introduction

Le recrutement dans le secteur de la Tech occupe aujourd'hui une place importante pour les entreprises, qui doivent identifier rapidement des profils développeurs adaptés à leurs besoins. Pourtant, les méthodes de sélection reposent encore souvent sur des critères très explicites : mots-clés présents dans les CV, intitulés de poste, diplômes, années d'expérience ou technologies mentionnées. Cette logique facilite le tri des candidatures, mais elle peut aussi écarter des profils pertinents, notamment juniors, autodidactes ou issus de parcours atypiques.

À ces limites s'ajoute la question des biais de recrutement. Certaines informations personnelles, comme le nom, l'âge, le genre ou l'origine supposée d'un candidat, peuvent influencer consciemment ou inconsciemment l'évaluation d'un profil. Dans un domaine où les compétences techniques et la capacité d'apprentissage sont essentielles, il devient donc nécessaire de proposer des outils capables de mieux valoriser le potentiel réel des candidats.

C'est dans ce contexte que nous avons conçu DevSpot, une plateforme web de recrutement dédiée aux métiers du développement et de l'IT. DevSpot ne se limite pas à un annuaire de profils : la plateforme intègre un système de matching sémantique permettant de comparer les offres et les profils au-delà des simples mots-clés, ainsi qu'une logique d'anonymisation destinée à limiter l'exposition des données personnelles lors de l'évaluation initiale.

Ce rapport présente la conception et la réalisation de DevSpot. Nous y détaillons le besoin ciblé, la problématique traitée, les choix méthodologiques, l'architecture de la solution, les fonctionnalités développées ainsi que les démarches de test et de validation mises en place. L'objectif est de montrer comment nous avons répondu à un besoin concret du recrutement tech, tout en intégrant des enjeux de qualité logicielle, de sécurité, d'équité et de démonstrabilité.

Besoin ciblé et problématique

Avec DevSpot, nous avons répondu à un besoin concret du recrutement tech : mieux identifier les profils développeurs pertinents tout en limitant certains biais présents dans les processus de sélection classiques. Les recruteurs s'appuient souvent sur des critères faciles

à filtrer, comme les technologies mentionnées, les intitulés de poste, les diplômes ou le nombre d'années d'expérience. Cette méthode permet un tri rapide, mais elle repose fortement sur la présence de mots-clés précis.

Cette logique peut écarter des candidats pertinents simplement parce qu'ils ne formulent pas leurs compétences avec les mêmes termes que ceux présents dans une offre. Elle désavantage aussi certains profils juniors, autodidactes ou issus de parcours atypiques, dont les compétences sont parfois acquises à travers des projets personnels, des formations courtes ou des expériences concrètes. À cela s'ajoute la question des biais liés aux données personnelles, comme le nom, l'âge, le genre ou l'origine supposée, qui peuvent influencer l'évaluation trop tôt dans le processus.

Nous avons donc construit DevSpot autour de la problématique suivante : Comment concevoir une plateforme de recrutement tech capable d'améliorer la pertinence de l'appariement entre offres et profils développeurs, tout en limitant les biais liés au filtrage lexical et à l'exposition des données personnelles ?

L'opportunité du projet repose sur sa capacité à combiner trois dimensions. Sur le plan technique, DevSpot utilise le matching sémantique pour comparer les offres et les profils au-delà des simples mots-clés. Sur le plan métier, la plateforme fournit aux recruteurs des outils de recherche, de contact et de suivi des candidats. Sur le plan éthique, elle intègre une logique d'anonymisation partielle afin de recentrer l'évaluation initiale sur les compétences, les expériences et les réalisations du candidat.

Motivation et opportunité

Nous avons choisi de développer DevSpot à partir d'un constat simple : dans le recrutement tech, il existe souvent un décalage entre les besoins des entreprises et la manière dont les profils développeurs sont évalués. Les recruteurs cherchent à identifier rapidement des candidats compétents, tandis que certains développeurs, notamment juniors ou issus de parcours atypiques, peinent à rendre visibles leurs compétences lorsqu'elles ne correspondent pas exactement aux critères classiques.

DevSpot représente donc une opportunité à la fois technique, métier et éthique. Sur le plan technique, le projet nous a permis d'explorer l'utilisation du matching sémantique dans un cas concret. Sur le plan métier, la plateforme apporte aux recruteurs un outil structuré pour rechercher, contacter et suivre les profils. Sur le plan éthique, l'anonymisation partielle permet de limiter l'exposition de certaines données personnelles lors de l'évaluation initiale.

Ainsi, DevSpot nous a permis de traiter un problème concret tout en mobilisant plusieurs compétences du Master : développement web, conception logicielle, gestion de projet, sécurité, qualité logicielle, documentation et intelligence artificielle.

Positionnement par rapport à l'existant

DevSpot s'inscrit dans un environnement où plusieurs types de solutions de recrutement existent déjà. Les jobboards, les réseaux professionnels, les ATS ou encore les CVthèques permettent aux recruteurs de publier des offres, de rechercher des candidats et de gérer certaines étapes du recrutement. Cependant, ces outils reposent encore souvent sur une logique de recherche classique, dans laquelle les profils sont triés à partir de mots-clés, de filtres ou de critères très explicites.

Les jobboards traditionnels permettent une diffusion large des offres d'emploi, mais ils ne garantissent pas toujours une analyse fine des compétences des candidats. Les recruteurs peuvent recevoir de nombreuses candidatures, sans forcément disposer d'outils capables d'évaluer la proximité réelle entre une offre et un profil. De leur côté, les candidats doivent souvent adapter leur CV aux mots-clés attendus pour être visibles.

Les réseaux professionnels, comme les plateformes de mise en relation, offrent une meilleure visibilité aux profils, mais ils exposent aussi beaucoup d'informations personnelles dès le premier contact. Cette visibilité peut être utile pour créer du réseau, mais elle ne répond pas entièrement à l'objectif d'une première évaluation centrée sur les compétences. Les informations personnelles, le parcours visible ou la présentation du profil peuvent influencer la perception du recruteur avant même l'analyse du contenu professionnel.

Les ATS, utilisés par les entreprises pour suivre les candidatures, sont davantage orientés vers la gestion interne du processus de recrutement. Ils permettent de centraliser les candidatures, de suivre les statuts et de gérer les échanges. Cependant, ils sont souvent

pensés pour organiser un pipeline de recrutement plutôt que pour valoriser des profils développeurs issus de parcours variés ou pour limiter les biais dès la première analyse.

Enfin, les CVthèques et outils de recherche de profils permettent de filtrer rapidement des candidats selon des critères précis. Leur limite principale reste la dépendance aux informations explicitement renseignées. Un candidat qui possède une compétence mais ne l'a pas formulée avec le bon terme peut être moins bien classé ou ne pas apparaître dans les résultats.

DevSpot se positionne comme une solution complémentaire à ces outils. Le projet ne cherche pas à remplacer l'ensemble des plateformes existantes, mais à proposer une approche plus ciblée pour le recrutement tech. Sa valeur ajoutée repose sur trois éléments principaux : la spécialisation autour des profils développeurs, l'utilisation du matching sémantique pour dépasser les simples mots-clés, et l'intégration d'une logique d'anonymisation pour réduire l'exposition des données personnelles lors de l'évaluation initiale.

Ainsi, DevSpot se distingue par sa volonté de combiner recherche de profils, suivi recruteur, messagerie, scoring de pertinence et approche éthique. La plateforme vise à offrir un outil plus adapté aux besoins des recruteurs, tout en permettant aux candidats de mieux faire reconnaître leurs compétences, leurs projets et leur potentiel.

Méthodologie et organisation

Nous avons réalisé DevSpot en binôme, avec une organisation progressive basée sur des sprints. Cette méthode nous a permis de découper le projet en étapes claires, en commençant par les fondations techniques avant d'intégrer les fonctionnalités métier, la messagerie, l'administration, le matching IA et les éléments de qualité logicielle.

Nous avons organisé notre travail autour d'un backlog de User Stories et de tickets GitHub. Chaque ticket correspondait à une tâche précise : création d'une entité, développement d'un formulaire, ajout d'une page, mise en place d'un contrôle d'accès, correction d'un bug ou intégration d'un service technique. Cette organisation nous a permis de suivre l'avancement du projet, de prioriser les fonctionnalités importantes et de garder une trace des décisions prises.

Le planning initial était structuré autour de plusieurs sprints : les premiers ont porté sur l'initialisation du projet, la base de données, l'authentification et les profils développeurs ; les suivants ont intégré l'espace recruteur, les offres d'emploi, la recherche, les favoris, l'administration, les notifications et la messagerie ; la dernière phase a été consacrée au matching IA, à la documentation, aux tests, à la CI/CD et à l'amélioration de l'interface.

Nous avons travaillé comme co-développeurs fullstack. Les tâches ont été réparties de manière collaborative entre la conception, le développement Symfony, les entités Doctrine, les contrôleurs, les formulaires, l'intégration Twig/Tailwind, la sécurité, les tests et la documentation. Cette organisation nous a permis de partager la charge de travail tout en conservant une vision commune de l'application.

Enfin, nous avons utilisé Git et GitHub pour le versionnage du code, le suivi des tickets et la centralisation des productions du projet. Le dépôt GitHub regroupe le code source, les livrables, la documentation technique et les instructions d'installation nécessaires à l'exécution de l'application.

Conception de la solution

Nous avons conçu DevSpot autour d'une architecture web structurée, composée d'une application Symfony principale, d'une base de données PostgreSQL, d'un service IA séparé et d'un système de messagerie en temps réel. Cette organisation nous a permis de séparer les responsabilités entre la logique métier, le stockage des données, le matching sémantique et les échanges entre utilisateurs.

L'application Symfony constitue le cœur du projet. Elle gère l'authentification, les rôles, les profils candidats, les profils recruteurs, les entreprises, les offres d'emploi, les favoris, les conversations, les notifications, les dashboards et l'administration. Les données sont stockées dans PostgreSQL et manipulées avec Doctrine ORM, ce qui nous a permis de structurer les principales entités métier et leurs relations.

Nous avons organisé la plateforme autour de trois espaces. L'espace candidat permet de créer un profil développeur complet avec les compétences, expériences, formations, technologies, liens externes et informations de visibilité. L'espace recruteur permet de rechercher des profils, gérer des offres, ajouter des favoris, contacter des candidats et suivre

les interactions. La gestion des offres inclut la création, la modification, la suppression, la fermeture et la réouverture, avec une règle de cohérence empêchant la republication d'une offre expirée sans mise à jour de sa date limite. L'espace administrateur permet de superviser les utilisateurs, les rôles, les statuts et les actions sensibles.

Le modèle de données reprend ces grands domaines fonctionnels. Il regroupe les comptes utilisateurs, les profils développeurs, les expériences, les formations, les compétences, les technologies, les profils recruteurs, les entreprises, les offres d'emploi, les favoris, les conversations, les messages, les notifications et les logs. Cette structure permet de couvrir les parcours principaux tout en conservant une séparation claire entre les données métier, les échanges utilisateurs et les données de traçabilité.

Nous avons également intégré un service IA séparé, développé avec FastAPI et Python. Ce service compare les offres d'emploi avec les profils développeurs afin de produire un score de pertinence. L'objectif est de dépasser la simple correspondance de mots-clés en analysant le sens des informations renseignées. Ce choix d'architecture nous a permis d'isoler les dépendances liées au traitement sémantique et de garder une application Symfony plus claire.

La messagerie interne repose sur Mercure afin d'apporter une dimension temps réel aux échanges entre candidats et recruteurs. Les notifications complètent ce fonctionnement en informant les utilisateurs des événements importants, tandis que les logs permettent de conserver une trace des actions sensibles, notamment dans l'espace d'administration.

Enfin, nous avons conçu les dashboards comme des points d'entrée adaptés à chaque rôle. Le dashboard candidat affiche l'état du profil, sa publication, ses messages, ses notifications et des recommandations d'amélioration. La visibilité du profil reste à 0 % tant que le profil est privé, même si certaines informations sont complétées. Le dashboard recruteur affiche les indicateurs de suivi des interactions : profils contactés, réponses reçues, favoris, conversations actives, conversations sans réponse, taux de réponse et profils liés à une offre. Le recruteur dispose d'un dashboard orienté suivi des interactions avec les candidats. L'administrateur dispose d'une vue de supervision sur les utilisateurs, les logs et les actions de gestion.

Réalisation / mise en œuvre

Nous avons réalisé DevSpot avec une stack technique cohérente avec les besoins du projet. Le cœur de l'application repose sur Symfony 7 et PHP 8, qui nous ont permis de structurer la logique métier, les routes, les contrôleurs, les formulaires, la sécurité et la gestion des rôles. Les interfaces ont été développées avec Twig et Tailwind CSS afin de conserver une intégration simple avec Symfony tout en gardant une interface personnalisable. Nous avons également travaillé l'adaptation responsive de l'interface. Certains tableaux ou blocs sont réorganisés sur mobile, les éléments trop lourds peuvent être masqués, et des boutons de retour facilitent la navigation dans les sous-pages comme les paramètres.

La base de données PostgreSQL est utilisée pour stocker l'ensemble des données de la plateforme : utilisateurs, profils développeurs, expériences, formations, compétences, recruteurs, entreprises, offres d'emploi, favoris, conversations, messages, notifications et logs. Nous avons utilisé Doctrine ORM pour créer les entités, gérer leurs relations et suivre l'évolution du schéma grâce aux migrations.

Nous avons développé les principaux parcours utilisateurs. Le candidat peut créer son compte, compléter son profil, ajouter ses expériences, formations, compétences et technologies, puis publier ou masquer son profil.

Le recruteur peut créer son profil, le rattacher à une entreprise, publier des offres, les modifier, les supprimer, les fermer ou les réouvrir. Une règle métier empêche la republication d'une offre fermée ou expirée tant qu'une nouvelle date limite valide n'a pas été renseignée. Le recruteur peut également rechercher des profils développeurs, les ajouter en favoris, les contacter et suivre ses interactions depuis son dashboard.

L'administrateur dispose d'un espace de gestion lui permettant de superviser les utilisateurs, les rôles, les statuts et les actions sensibles.

Nous avons également mis en place une messagerie interne entre candidats et recruteurs. L'intégration de Mercure permet d'améliorer la fluidité des échanges grâce au temps réel. Les notifications complètent ce système en informant les utilisateurs des événements importants, tandis que les logs assurent la traçabilité des actions sensibles. Elles sont

consultables depuis un espace dédié. Elles peuvent être marquées comme lues ou supprimées, avec une interface adaptée aux écrans desktop et mobile.

Le service de matching IA a été intégré sous forme de service séparé avec FastAPI et Python. Il compare les offres d'emploi avec les profils développeurs à partir des compétences, des expériences et du sens global des informations renseignées. Cette approche permet de produire un score de pertinence exploitable par le recruteur, sans se limiter à une simple recherche de mots-clés.

Nous avons aussi intégré une logique d'anonymisation partielle afin de limiter l'exposition des données personnelles lors de l'évaluation initiale des profils. Cette anonymisation permet de recentrer l'analyse sur les compétences, les expériences, les formations et les réalisations du candidat.

Enfin, nous avons mis en place une documentation technique avec MkDocs ainsi qu'un environnement Docker pour faciliter l'installation du projet. GitHub Actions nous a permis d'automatiser certains contrôles qualité, notamment les tests, l'analyse du code et la vérification du style. Ces éléments contribuent à rendre le projet plus maintenable, plus fiable et plus facile à exécuter.

Tests, validation et limites

Nous avons validé DevSpot à plusieurs niveaux afin de vérifier le bon fonctionnement des parcours principaux, la cohérence technique de l'application et la fiabilité des fonctionnalités développées. L'objectif était de s'assurer que la plateforme pouvait être utilisée correctement par les trois rôles principaux : candidat, recruteur et administrateur.

Nous avons d'abord vérifié les parcours fonctionnels essentiels. Le parcours candidat a été testé depuis l'inscription jusqu'à la publication du profil, en passant par l'ajout des expériences, formations, compétences et technologies. Le parcours recruteur a été vérifié à travers la création du profil recruteur, le rattachement à une entreprise, la publication d'offres, la recherche de profils, l'ajout de favoris et l'envoi de messages. Le parcours administrateur a été validé à travers la gestion des utilisateurs, des rôles, des statuts et la consultation des logs.

Nous avons également contrôlé la sécurité des accès. Les pages privées sont réservées aux utilisateurs authentifiés, et les fonctionnalités sont limitées selon les rôles. Un candidat ne peut pas accéder aux fonctionnalités recruteur, un recruteur ne peut pas accéder aux fonctionnalités administrateur, et un visiteur non connecté est redirigé vers les pages publiques ou vers la connexion.

Sur le plan technique, nous avons utilisé des tests unitaires et fonctionnels afin de vérifier les services, les contrôleurs et certains parcours critiques. Les contrôles de qualité, comme PHP CS Fixer et l'analyse statique, ont permis d'améliorer la cohérence du code et de limiter les erreurs de structure. GitHub Actions a été utilisé pour automatiser une partie de ces vérifications et sécuriser l'évolution du projet.

Nous avons aussi validé la messagerie interne et les notifications. Les échanges entre candidats et recruteurs ont été testés afin de vérifier la création des conversations, l'envoi des messages, l'affichage de l'historique et la réception des informations utiles. L'intégration de Mercure a permis de rendre ces échanges plus dynamiques grâce au temps réel.

Enfin, nous avons évalué le service de matching IA en comparant les résultats obtenus avec une logique plus classique de recherche par mots-clés. Cette validation nous a permis de vérifier que le score produit était exploitable par le recruteur et qu'il permettait de mieux valoriser certains profils lorsque les compétences étaient formulées différemment. L'anonymisation partielle a également été vérifiée afin de s'assurer que les données personnelles non nécessaires à l'évaluation initiale étaient limitées.

Ces tests et vérifications nous ont permis de valider que DevSpot répondait aux objectifs principaux du projet : proposer une plateforme fonctionnelle, sécurisée, documentée et capable de démontrer l'intérêt d'un recrutement tech plus sémantique et plus équitable.

Limites et perspectives

Même si DevSpot répond aux objectifs principaux fixés au départ, le projet peut encore évoluer. Les fonctionnalités essentielles sont opérationnelles, mais certaines améliorations pourraient renforcer la qualité de la plateforme, l'expérience utilisateur et la précision du matching.

Une première perspective concerne le service de matching IA. Le score actuel permet de comparer les offres et les profils à partir des compétences, des expériences et de la proximité sémantique. Il pourrait être enrichi avec davantage de données, un jeu de test plus large et une pondération plus fine des critères. Cela permettrait d'obtenir des résultats plus précis et de mieux expliquer pourquoi un profil est proposé à un recruteur.

Une deuxième évolution possible concerne l'import automatique de CV. Pour l'instant, le candidat renseigne lui-même les informations de son profil. À terme, la plateforme pourrait extraire automatiquement certaines données depuis un CV PDF, puis laisser le candidat les vérifier et les corriger. Cela rendrait la création de profil plus rapide.

Le dashboard recruteur intègre déjà des indicateurs de suivi comme les profils contactés, les réponses, les favoris, les conversations actives, les conversations sans réponse, le taux de réponse et les profils liés à une offre. Une perspective d'évolution serait d'aller plus loin avec des statistiques historiques, des graphiques d'évolution ou des recommandations automatiques de relance. Ces indicateurs permettraient de renforcer la dimension pilotage du recrutement.

Enfin, l'interface pourrait continuer à être améliorée afin de rendre les parcours encore plus fluides. Des tests utilisateurs plus poussés permettraient d'identifier les points de friction et d'améliorer l'expérience candidat, recruteur et administrateur.

Ces perspectives ne remettent pas en cause le fonctionnement actuel de DevSpot. Elles montrent surtout que le projet dispose d'une base solide, sur laquelle de nouvelles fonctionnalités peuvent être ajoutées progressivement.

Conclusion

DevSpot nous a permis de concevoir et de réaliser une plateforme de recrutement tech complète, centrée sur la mise en relation entre développeurs et recruteurs. Le projet répond à une problématique concrète : améliorer l'identification des profils pertinents tout en limitant les biais liés au filtrage par mots-clés et à l'exposition précoce de données personnelles.

Nous avons développé une application structurée autour de trois espaces principaux : candidat, recruteur et administrateur. Les candidats peuvent créer et valoriser leur profil professionnel, les recruteurs peuvent rechercher, sauvegarder, contacter et suivre les profils, tandis que les administrateurs disposent d'outils de supervision et de traçabilité. L'intégration de la messagerie, des notifications, des dashboards, de l'anonymisation et du matching IA permet à DevSpot de dépasser le fonctionnement d'un simple annuaire de profils.

Sur le plan technique, le projet nous a permis de mobiliser plusieurs compétences : développement Symfony, conception de base de données, sécurité, intégration de services, documentation, tests, CI/CD et intelligence artificielle. L'architecture retenue, avec une application Symfony principale et un service IA séparé, offre une base claire et évolutive.

Au final, DevSpot constitue une solution fonctionnelle et démontrable, qui combine des enjeux techniques, métier et éthiques. Le projet montre qu'il est possible de proposer une approche plus pertinente du recrutement tech, en valorisant davantage les compétences réelles des candidats et en fournissant aux recruteurs des outils de suivi plus adaptés.

Annexes

Annexe 1 - Captures des dashboards et page d'accueil

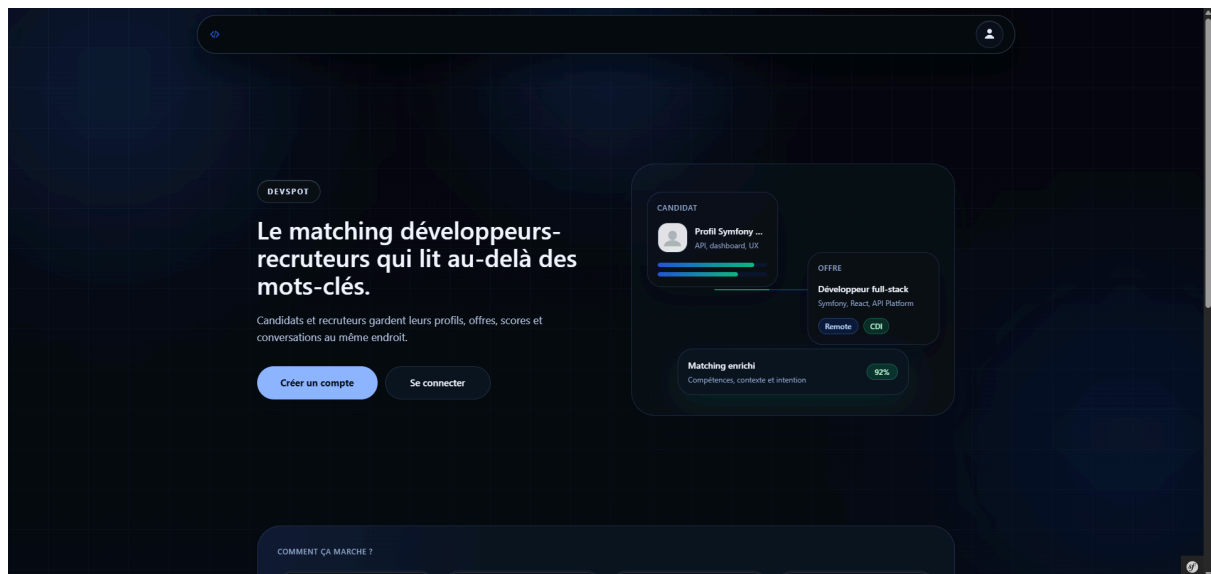


Figure 1 — Page d'accueil pour les utilisateurs non connectés.

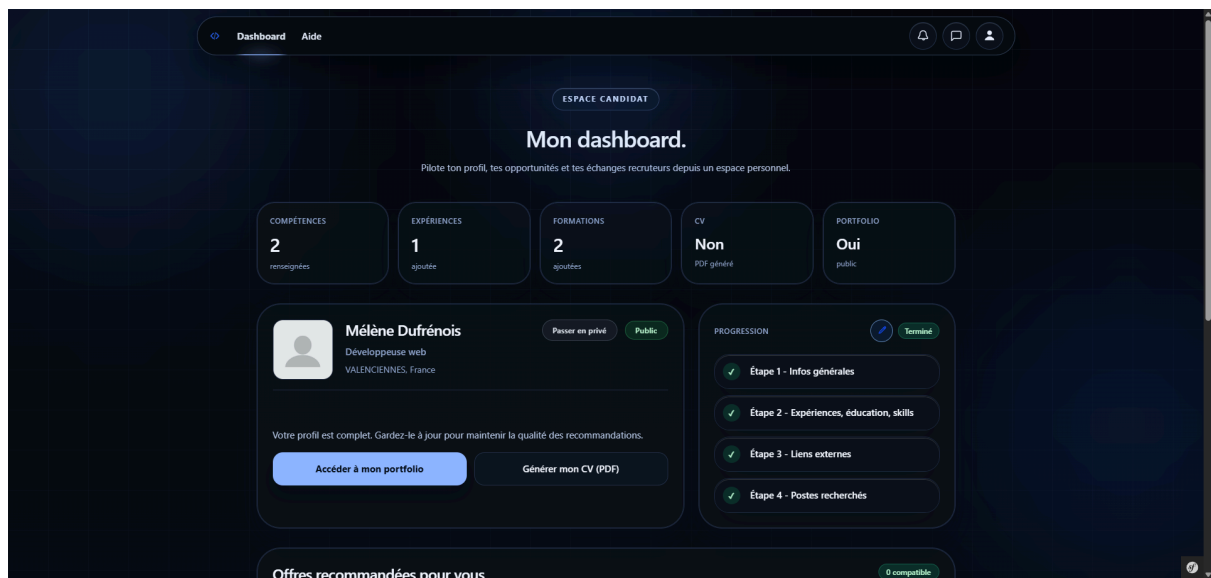


Figure 2 — Dashboard candidat avec état du profil et recommandations.

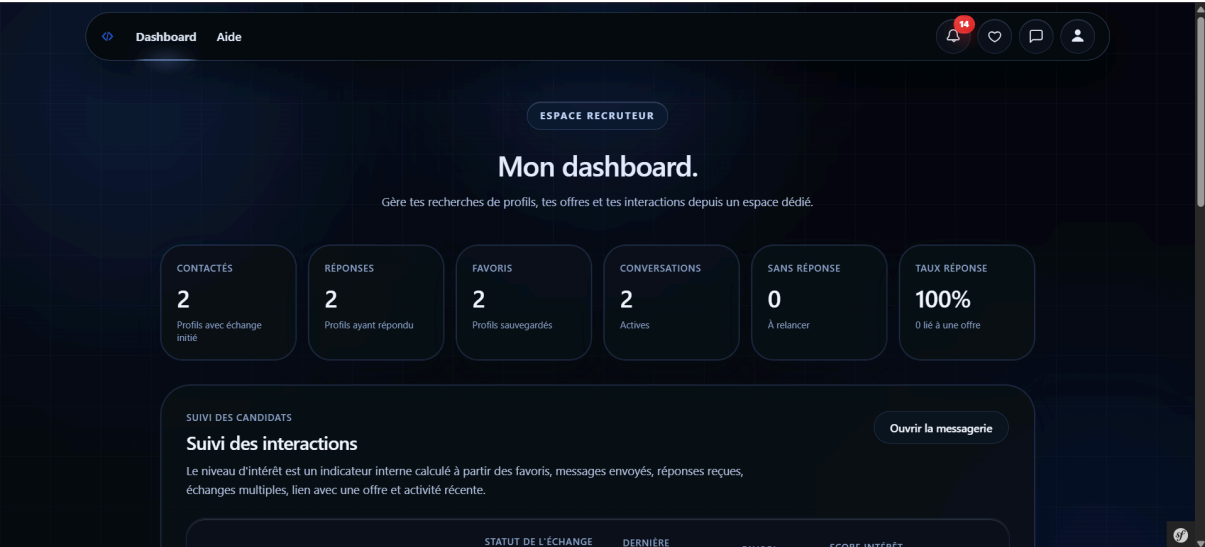


Figure 3 – Dashboard recruteur orienté suivi des interactions.

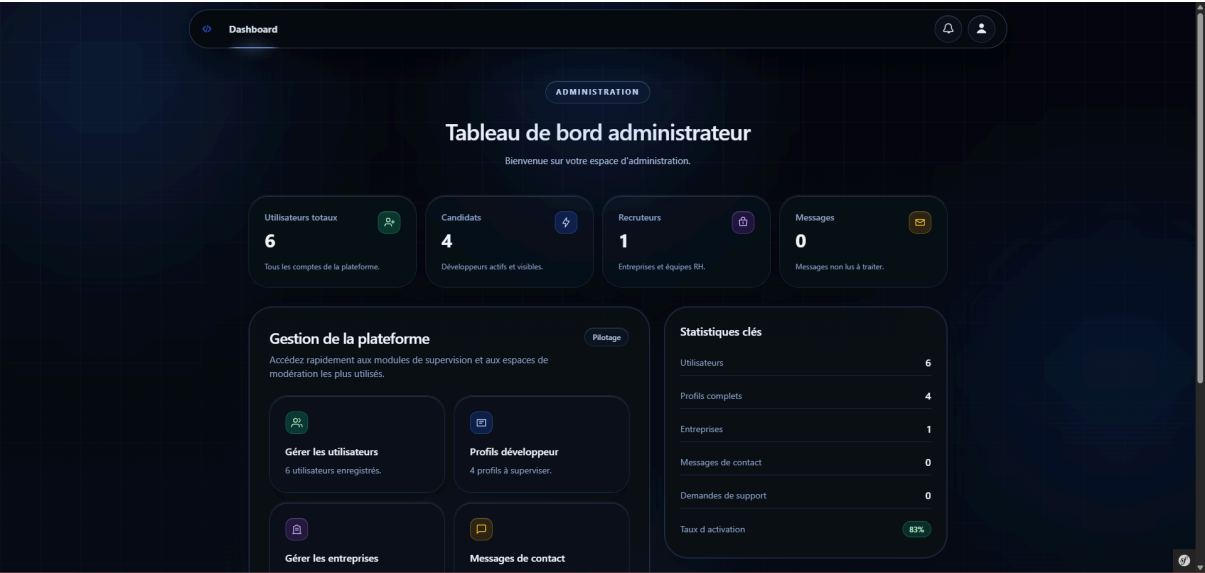


Figure 4 – Tableau de bord administrateur et supervision de la plateforme.

Annexe 2 - Captures des fonctionnalités principales de l'application

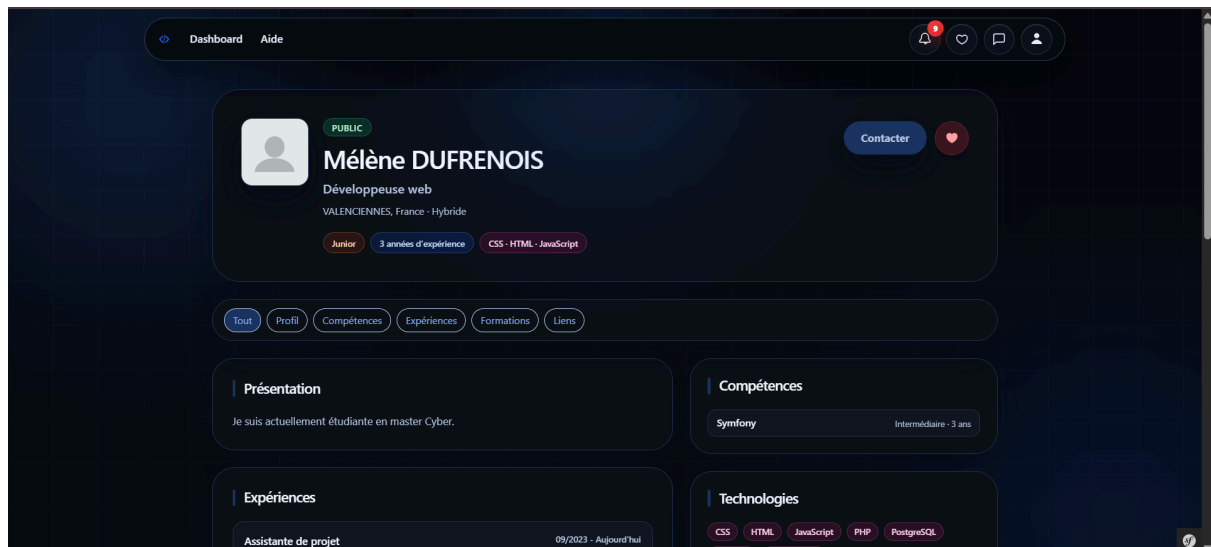


Figure 5 – Profil portfolio d'un candidat.

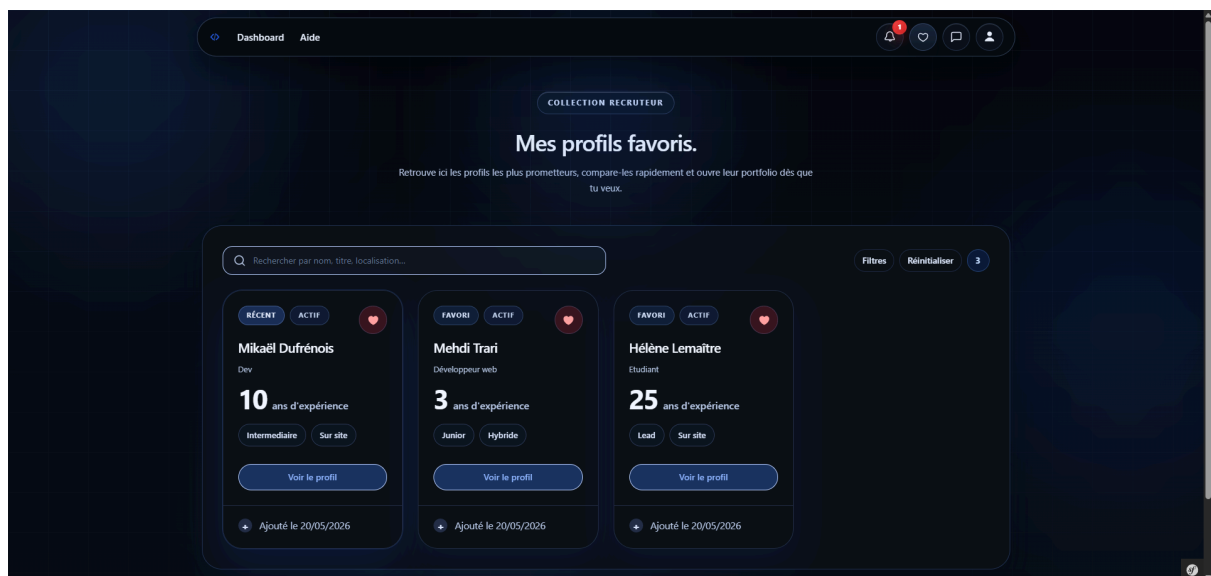


Figure 6 – Candidats favoris d'un recruteur.

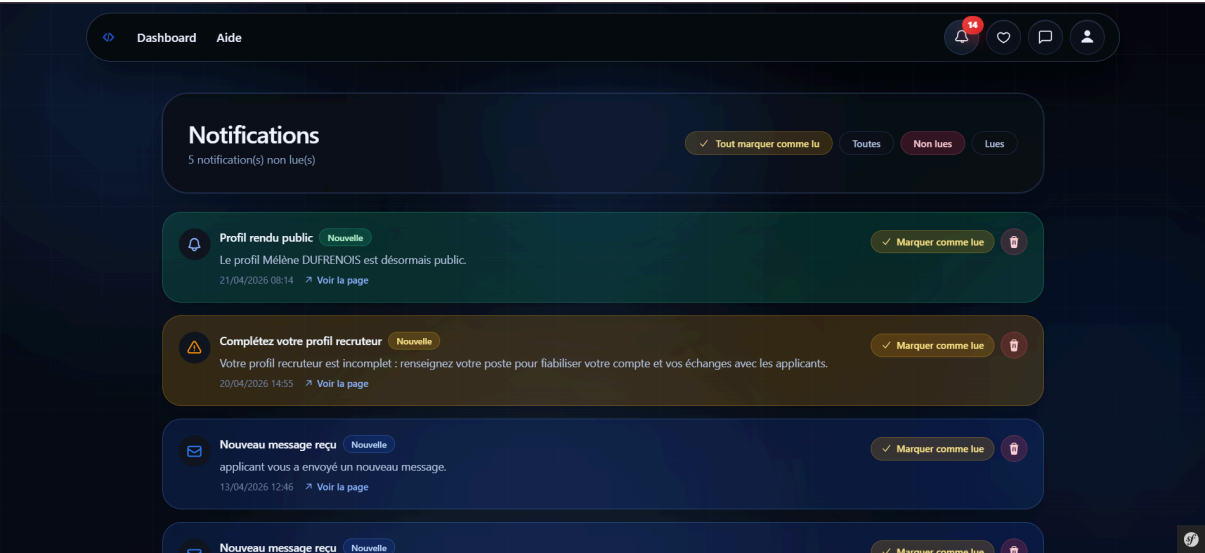


Figure 7 – Notifications.

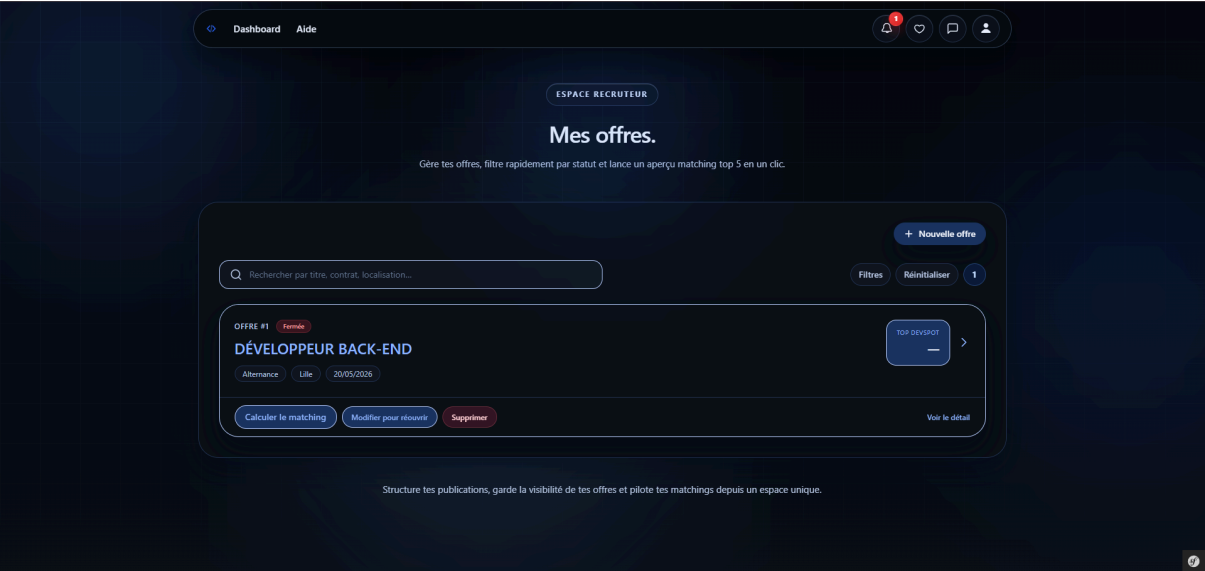
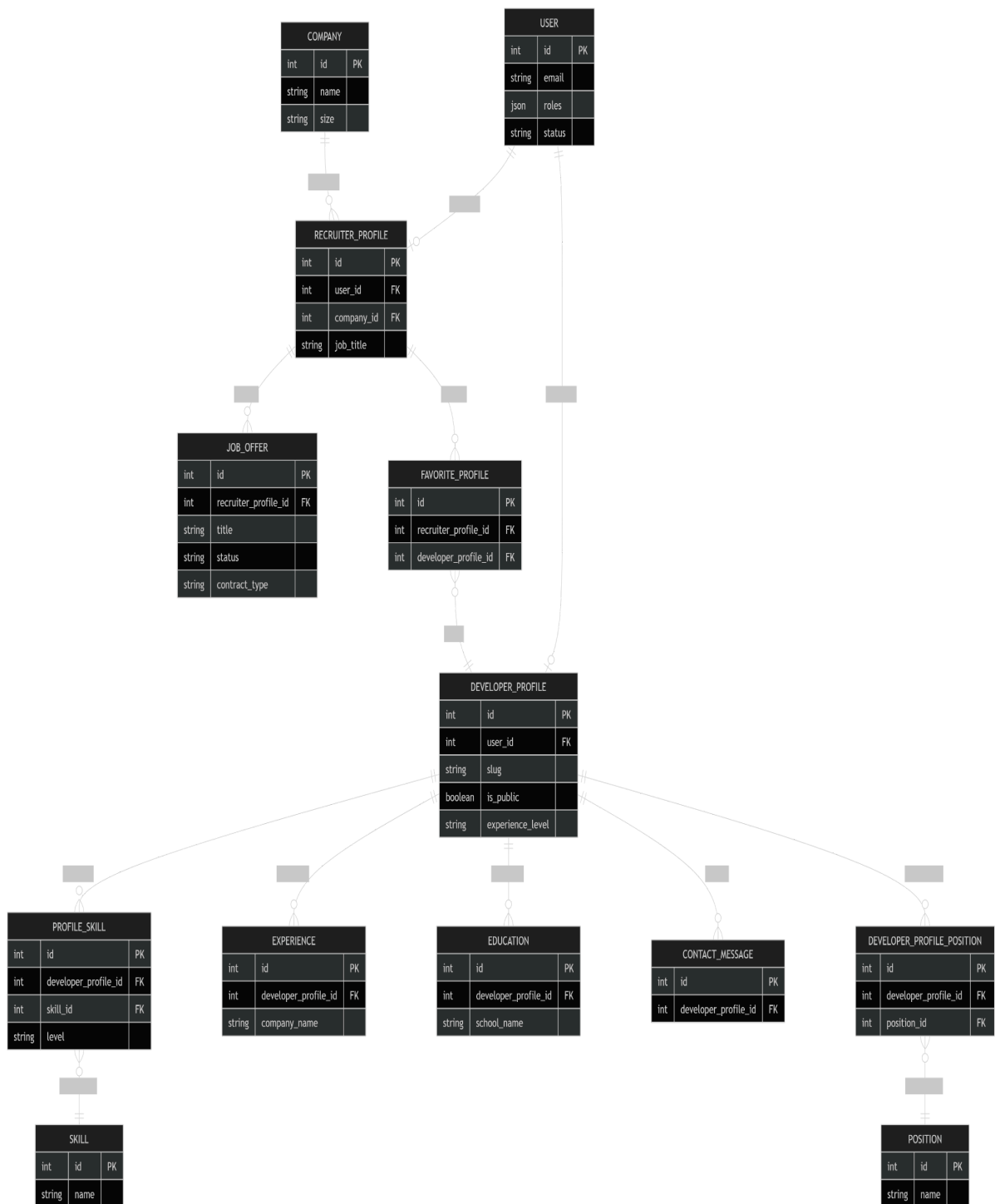


Figure 8 – Offres créés par un recruteur.

Annexe 3 - Modèle de données / base de données



Annexe 4 - Documentation technique / GitHub

(Disponible en détail dans notre repository ci-dessous)

Repository Github : <https://github.com/MehdiTrari/DevSpot>

Documentation complète : <https://github.com/MehdiTrari/DevSpot/tree/develop/docs>

Prérequis :

Docker + Docker Compose

PHP 8.2+

Composer

Symfony CLI optionnel

Commandes d'installation :

```
git clone https://github.com/MehdiTrari/DevSpot.git
```

```
cd DevSpot
```

```
composer install
```

Lancement Docker :

```
docker compose up -d --build
```

```
docker compose exec app php bin/console doctrine:migrations:status
```

```
docker compose exec app php bin/console doctrine:migrations:migrate --no-interaction
```

Liens utiles :

Application : <http://devspot.localhost>

Application fallback : <http://localhost:8000>

Documentation : <http://localhost:8002/DevSpot/>

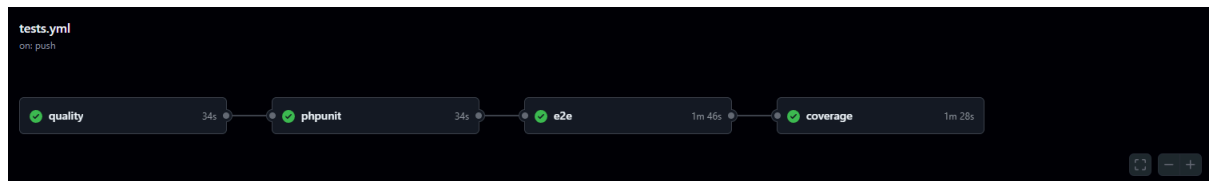
Adminer : <http://localhost:8081>

Mailpit : <http://localhost:8025>

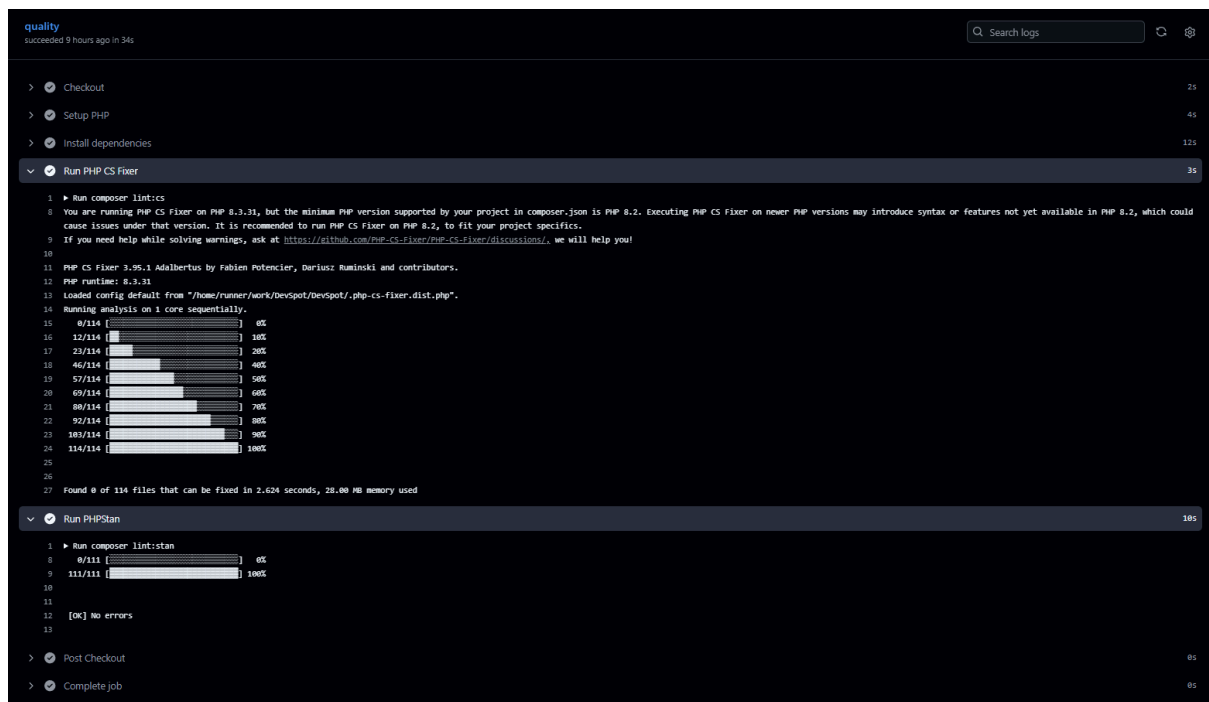
Service ML : <http://localhost:8001>

Annexe 5 - Tests et qualité

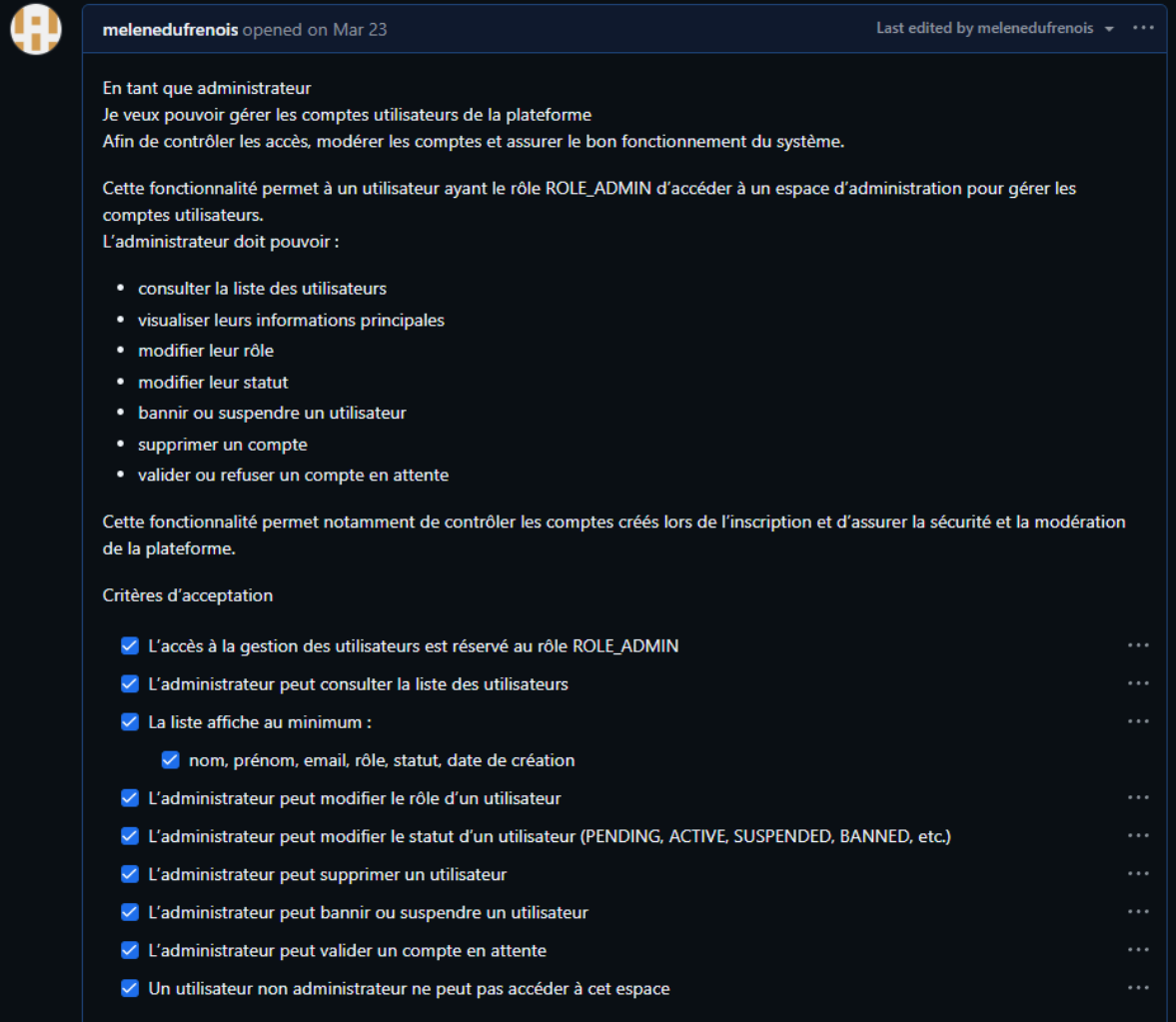
vue globale de la pipeline de test :



Exemple du job “quality” qui lance php stan et cs fixer :



Annexe 6 - Suivi projet / organisation



melenedufrenois opened on Mar 23 Last edited by melenedufrenois

En tant que administrateur
Je veux pouvoir gérer les comptes utilisateurs de la plateforme
Afin de contrôler les accès, modérer les comptes et assurer le bon fonctionnement du système.

Cette fonctionnalité permet à un utilisateur ayant le rôle `ROLE_ADMIN` d'accéder à un espace d'administration pour gérer les comptes utilisateurs.
L'administrateur doit pouvoir :

- consulter la liste des utilisateurs
- visualiser leurs informations principales
- modifier leur rôle
- modifier leur statut
- bannir ou suspendre un utilisateur
- supprimer un compte
- valider ou refuser un compte en attente

Cette fonctionnalité permet notamment de contrôler les comptes créés lors de l'inscription et d'assurer la sécurité et la modération de la plateforme.

Critères d'acceptation

- ☒ L'accès à la gestion des utilisateurs est réservé au rôle `ROLE_ADMIN`
- ☒ L'administrateur peut consulter la liste des utilisateurs
- ☒ La liste affiche au minimum :
 - ☒ nom, prénom, email, rôle, statut, date de création
- ☒ L'administrateur peut modifier le rôle d'un utilisateur
- ☒ L'administrateur peut modifier le statut d'un utilisateur (`PENDING`, `ACTIVE`, `SUSPENDED`, `BANNED`, etc.)
- ☒ L'administrateur peut supprimer un utilisateur
- ☒ L'administrateur peut bannir ou suspendre un utilisateur
- ☒ L'administrateur peut valider un compte en attente
- ☒ Un utilisateur non administrateur ne peut pas accéder à cet espace

Figure 13 — Issue de GitHub.

Branch	Updated	Check status	Behind	Ahead	Pull request
feature/US56-2	2 hours ago	✗ 3 / 9	2	275	#125
develop	2 hours ago	✗ 4 / 11	2	271	#124
feature/livrable_final	7 hours ago	✗ 0 / 4	2	273	
feature/US56	7 hours ago	✓ 9 / 9	2	268	#123
feature/US53	yesterday	✓ 9 / 9	2	257	#120
preprod	yesterday	✓ 4 / 4	2	250	
feature/US28	2 weeks ago	✓ 4 / 4	2	245	
feature/US43	2 weeks ago	✓ 9 / 9	2	243	#118
fix/BUG1	2 months ago	✗ 0 / 2	2	90	

Figure 14 — Extrait de nos branches.

Commits on Apr 21, 2026		
🌟 FAQ & support	melenedufrenois committed on Apr 21 · 🚫 2 / 4	9623c56 <>
Merge pull request #106 from MehdiTrari/feature/US48	Verified melenedufrenois authored on Apr 21 · 🚫 4 / 6	0b8ba1e <>
Merge branch 'develop' of https://github.com/MehdiTrari/DevSpot into feature/US48	melenedufrenois committed on Apr 21 · ✅ 9 / 9	f4ebacb <>
✅ fix test e2e	melenedufrenois committed on Apr 21 · 🚫 7 / 9	757bd9c <>
Merge pull request #107 from MehdiTrari/feature/US49	Verified MehdiTrari authored on Apr 21 · 🚫 4 / 6	16f41d6 <>
🐛 cs fixer	melenedufrenois committed on Apr 21 · 🚫 7 / 9	7c144c5 <>
Merge branch 'develop' of https://github.com/MehdiTrari/DevSpot into feature/US48	melenedufrenois committed on Apr 21	3676b64 <>
🐛 apply cs fixer	melenedufrenois committed on Apr 21 · ✅ 4 / 4	4a43f57 <>
Merge remote-tracking branch 'origin/develop' into feature/US49	Copilot and MehdiTrari committed on Apr 21 · ✅ 9 / 9	5894c85 <>
📄 report	melenedufrenois committed on Apr 21 · 🚫 4 / 8	c2d807c <>
📄 add Mkdocs and update all docs + dockerised all project	MehdiTrari committed on Apr 21 · ✅ 4 / 4	5ae0a3b <>
🐛 hover for gestion menu users	melenedufrenois committed on Apr 21 · 🚫 4 / 8	9c5e82a <>
🐛 modal message for users		04e4d44 <>

Figure 15 — Extraits de nos commits.